SeniorFullStack Skill – Ready-to-Deploy Claude/Kimi Native Package

I've reviewed the attached PDF (`context SeniorFullStack skill.pdf`) and the modified version you provided. The core idea is excellent: package 15+ years of **monolithic + decentralized** Django + React/TS expertise into a clean, Claude-native (and Kimi-compatible) skill hierarchy.

I've **polished the entire specification** for clarity, grammar, and consistency, properly integrating both monolithic and decentralized architectures (e.g., traditional monoliths, microservices, DDD, event-driven, or hybrid distributed setups). Below is the **production-ready** `SKILL.md` plus the exact folder structure you can copy-paste into `.claude/skills/senior-fullstack/` (or your project's equivalent).

1. SKILL.md (copy this file exactly)

```Markdown
---
name: SeniorFullStack
description: 15+ years of monolithic AND decentralized Django + React/TS
expertise for automated codebase analysis, architecture scoring, security review,
and production-ready scaffolding of Django backends + React/TS frontends.
version: 1.0.0
capabilities:
  - codebase_analysis
  - hierarchy_generation
  - security_review
  - monolith_and_decentralized_scaffolding
---

# SeniorFullStack Skill

**Purpose**
Automatically analyze any Django + React/TS codebase (monolithic or
decentralized) and emit a complete, production-grade folder hierarchy with
boilerplate, best-practice patterns, security hardening, and validation scripts.

**How to use**
1. Attach your existing codebase (or describe requirements).
2. The skill will:
   - Parse the current structure (using `scripts/analyze.py`)
```

```
   - Parse the current codebase (using `scripts/analyze.py`)
   - Score against Django/React best practices, OWASP, performance, and
scalability
   - Generate a validated `hierarchy/` output (monolith or
decentralized/microservices layout)
   - Return rationale + next-step recommendations

**Capabilities triggered by user intent**
- `codebase_analysis` → runs `analyze.py`
- `hierarchy_generation` → runs `generate_hierarchy.py`
- `security_review` → applies OWASP Django checklist + React patterns
- `monolith_and_decentralized_scaffolding` → chooses or combines monolith vs
decentralized templates

**Supported architectures**
- **Monolithic** – classic Django monolith with apps/, core/, services/
- **Decentralized / Microservices** – multiple bounded-context services, API
gateways, event-driven patterns, shared libraries
- **Hybrid** – monolith with clear service-extraction paths

**Output format**
- Validated folder tree (with boilerplate files)
- `README.md` explaining the chosen architecture
- `production-readiness.md` checklist with status
- Diff-style rationale for every major decision

**References & Templates** (loaded on-demand via progressive disclosure)
- `references/` – battle-tested markdown guides
- `assets/templates/` – ready-to-copy Django monolith + decentralized service
templates + React/TS feature-based frontend
- `scripts/` – Pydantic-validated Python tools
- `checklists/` – production, performance, security, and scalability checklists

**Security & Governance**
All templates are SHA256-signed (see `references/integrity.md`). Scripts run in
sandbox. No proprietary code is logged or persisted.

**Versioning & Updates**
Skill is Git-backed. Register in your project's `.claude/skills/` for team
sharing.
```

2. Exact Folder Hierarchy to Create

```text
.claude/skills/senior-fullstack/
├── SKILL.md                          # ← the file above
│
├── references/
│   ├── django_best_practices.md
│   ├── react_ts_patterns.md
│   ├── integration_patterns.md       # monolith ↔ decentralized communication
│   ├── owasp_django.md
│   └── integrity.md                  # SHA256 signatures + verification script
│
├── assets/
│   └── templates/
│       ├── django_monolith/          # classic monolith
│       │   ├── apps/
│       │   ├── core/
│       │   ├── services/
│       │   ├── settings/
│       │   └── urls/
│       │
│       ├── django_decentralized/     # microservices / bounded contexts
│       │   ├── service1/             # example bounded context
│       │   ├── service2/
│       │   ├── shared/
│       │   ├── api_gateway/
│       │   └── docker-compose.yml
│       │
│       └── react_frontend/           # feature-based TS + Vite
│           ├── src/
│           │   ├── features/
│           │   ├── shared/
│           │   ├── lib/
│           │   └── routing/
│           └── vite.config.ts
│
├── scripts/
│   ├── analyze.py                    # Pydantic-validated codebase parser
│   └── generate_hierarchy.py         # outputs production folder tree +
boilerplate
│
├── checklists/
```

```
│   ├── production_readiness.md
│   ├── performance_scaling.md
│   └── security_owasp.md
│
└── .skill-metadata.yaml              # optional: for hot-reload / meta-agents
```

Next Steps I Recommend

1. **Drop the files** into `.claude/skills/senior-fullstack/` (or your project's skill folder).
2. Let me know if you want me to **generate the full content** of any of the supporting files right now:
   - `references/django_best_practices.md`
   - `references/react_ts_patterns.md`
   - `scripts/analyze.py` & `generate_hierarchy.py`
   - Template boilerplates for monolith or decentralized Django services
   - Or the complete `assets/templates/` tree with real starter files
3. Once the skill is installed, you can simply say:

   > *"Analyze my current Django/React codebase and scaffold a production-ready decentralized version"*
   > *(or monolithic, hybrid, etc.)*

Would you like me to generate the **full set of reference files and scripts** now, or would you prefer to start with a specific part (e.g. just the Django decentralized template)? Just tell me your current project type (pure monolith, moving toward microservices, etc.) and I'll deliver exactly what you need.